Rodrigo Fernandez

Senior Go / React Full Stack Engineer
Katy, Texas | +1 (407) 801-1287 | www.linkedin.com/in/rodrigobermejo-fernández-9a5a0664 | rodrigobfdev@gmail.com

Summary

Senior Go + React Full Stack Engineer with enterprise experience across cloud infrastructure, SaaS, fintech, and digital commerce, building scalable APIs with **Go 1.18–1.22**, **Gin**, **gRPC**, **PostgreSQL 12–16**, and modern frontends with **React 17–18**, **TypeScript 4.x–5.x**, and **Next.js 12–14**. Specialized in migrating legacy Node.js services to Go microservices, optimizing high-traffic dashboards, fixing production latency and concurrency issues, and delivering secure cloud-native platforms with Docker, Kubernetes, AWS, CI/CD, and observability. Strong at turning product requirements into reliable features while reducing deployment risk with pragmatic engineering decisions.

Skills

- **Backend: Go 1.18–1.22**, Gin, Echo, Fiber, gRPC, REST API, GraphQL, Node.js 14–20, Express.js, NestJS, Microservices, Event-Driven Architecture, WebSocket, JWT, OAuth2, RBAC
- **Frontend: React 17–18**, TypeScript 4.x–5.x, JavaScript ES6+, Next.js 12–14, Redux Toolkit, React Query, Zustand, Material UI, Tailwind CSS, Ant Design, HTML5, CSS3
- **Database: PostgreSQL 12–16**, MySQL 8.x, MongoDB 4.x–6.x, Redis 6.x–7.x, SQL Optimization, Indexing, Query Tuning, Data Migration
- **Cloud & DevOps: AWS**, ECS, EKS, Lambda, S3, RDS, CloudWatch, Docker, Kubernetes 1.22–1.28, Terraform 1.x, GitHub Actions, GitLab CI/CD, Nginx
- **Testing & Quality:** Go Test, Jest, React Testing Library, Cypress, Playwright, Postman, Swagger/OpenAPI, SonarQube, ESLint, Prettier
- **Architecture & Practices:** Clean Architecture, Domain-Driven Design, CI/CD, Agile Scrum, Observability, Performance Optimization, Secure API Design, Code Review

Experience

Lightedge — Senior Software Engineer *(Apr 2020 – Mar 2026)*

- Developed **Go 1.16–1.19 backend services** and **React 17 applications** for SaaS and fintech clients, delivering secure account, payment, and reporting features with production-grade reliability.
- Migrated monolithic Express.js APIs into smaller **Go REST services** with PostgreSQL and Redis, reducing API timeout errors by **34%** during peak customer activity.
- Built complex React screens using **TypeScript 4.x**, Redux Toolkit, and Material UI, improving form validation, error handling, and user experience across customer management workflows.
- Fixed a major race condition in a wallet transaction service by introducing database row locking, idempotent request handling, and safer retry logic in the Go payment flow.
- Implemented new reporting features with paginated APIs, CSV export, and role-based access, helping business users process financial data **29% faster** without manual spreadsheet work.
- Improved frontend performance by splitting large React bundles, lazy loading heavy modules, and removing unnecessary re-renders from dashboard components.
- Created automated backend tests with **Go Test** and integration test containers, increasing confidence in releases and reducing regression defects by **26%**.

- Integrated third-party payment, email, and identity APIs with secure secret handling, request validation, and structured error mapping for frontend display.
- Refactored old React class components into modern functional components with hooks, making the frontend codebase easier to maintain and safer for new feature delivery.
- Supported Kubernetes-based deployment pipelines by improving Docker images, health checks, readiness probes, and environment-specific configuration management.
- Worked closely with product owners and QA engineers to convert unclear requirements into testable stories, stable API contracts, and realistic sprint deliverables.

NIX United —Software Engineer *(Oct 2016 – Mar 2020)*

- Developed backend services for fintech and healthcare clients using **Go 1.11–1.16**, PostgreSQL, Redis, RabbitMQ, and Docker, with strong focus on API reliability and maintainable service design.
- Migrated legacy PHP and Node.js backend modules into Go services where performance, concurrency, and simpler deployment artifacts were important for client-facing workloads.
- Implemented payment reconciliation workflows with Go workers, RabbitMQ queues, retry policies, and database-level idempotency, reducing duplicate transaction defects by **29%**.
- Fixed a difficult memory growth issue in a long-running Go service by profiling heap allocations, reducing unnecessary object retention, and improving batch processing logic.
- Built secure REST APIs with middleware for authentication, request validation, rate limiting, structured errors, and audit logging across regulated business workflows.
- Improved PostgreSQL query performance by normalizing overloaded tables, tuning indexes, and separating read-heavy reporting queries from transactional paths.
- Created Docker-based local development environments that helped engineers reproduce production-like issues faster and reduced onboarding setup time by **40%**.
- Added integration tests around critical billing and account-management paths using Go test suites, mocks, test containers, and seeded database fixtures.
- Implemented background notification services with retry handling, timeout control, and message deduplication to avoid repeated customer alerts during provider instability.
- Collaborated with frontend and QA teams to clarify API contracts, reduce breaking changes, and deliver backend features that matched real product behavior.
- Supported release stabilization by reviewing logs, fixing flaky integration tests, and improving rollback instructions before client production deployments.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Built **React 16–17** customer portals and **Go 1.13–1.16** backend APIs for digital commerce platforms, supporting product catalog, checkout, order tracking, and admin reporting features.
- Implemented REST APIs with Gin, PostgreSQL, and Redis caching, improving product search and order lookup response time by **33%** under higher traffic.
- Migrated several jQuery-heavy pages into reusable **React components**, improving maintainability and reducing UI bugs caused by duplicated DOM manipulation logic.
- Resolved checkout failures caused by inconsistent inventory updates by adding transaction-safe stock reservation logic and better API-level validation.
- Created responsive frontend layouts with React, TypeScript, and Material UI, improving mobile usability for customer-facing commerce pages.
- Implemented admin dashboards for order operations, refunds, and customer support workflows, reducing manual support handling time by **24%**.
- Improved database queries by adding indexes, rewriting slow joins, and separating reporting queries from transactional API paths.

- Added structured logging and API error tracking, helping the team identify failed payment callbacks and retry problems much faster during incidents.
- Worked flexibly across frontend, backend, and deployment tasks, supporting urgent bug fixes while continuing planned feature development.
- Collaborated with designers and stakeholders to convert business workflows into reliable application screens, clean API payloads, and practical release plans.

Aurio — Junior Software Developer *(Oct 2012 – Dec 2013)*

- Developed early **React 15–16** interface components and backend API features for internal business applications, focusing on usability, form workflows, and data accuracy.
- Supported backend services written in Go and Node.js by fixing API validation issues, improving request handling, and creating clearer error responses.
- Migrated legacy JavaScript pages into component-based React screens, reducing repeated UI code and improving consistency across internal tools.
- Fixed production bugs related to incorrect date filtering, empty API responses, and broken pagination in customer activity reports.
- Implemented role-based UI visibility rules so internal users could access only the correct modules based on permissions and team responsibility.
- Created reusable form components with validation, loading states, and error display patterns, improving the reliability of data-entry workflows.
- Improved SQL queries for reporting pages by removing unnecessary joins and adding safer filters for large datasets.
- Participated in code reviews, QA verification, and release preparation, building strong habits around maintainability and production safety.
- Learned to adapt quickly between frontend tasks, backend bug fixes, and deployment support while contributing to stable application delivery.

Microsoft — Software Developer Intern *(Jun 2012 – Sep 2012)*

- Supported internal engineering tools using **C#**, **.NET Framework 4.0–4.5**, **ASP.NET MVC 3–4**, JavaScript, jQuery, and SQL Server, gaining practical experience in enterprise software quality and structured development workflows.
- Assisted senior engineers with debugging UI defects, validation issues, and data-handling problems, documenting root causes clearly so fixes could be reviewed and released with lower regression risk.
- Built small internal web modules for tracking records, reviewing status changes, and improving team visibility, applying clean HTML, CSS, JavaScript, and backend integration patterns.
- Resolved early-stage bugs related to inconsistent form validation and SQL query results by reproducing issues locally, checking logs, and working with mentors to apply safer input-handling rules.
- Learned professional engineering practices around code reviews, source control, test cases, documentation, and release discipline, which later became the foundation for stronger **React.js** and full-stack delivery.

Education

Texas State University
Bachelor's Degree in Computer Science (*Sep 2008 – May 2012)*